If you have registered your team through the form already, the exercise repository will appear under https://github.com/dl-freiburg. If you have not registered your team yet, register through this link: https://script.google.com/macros/s/AKfycbydG3qJpr16oy0wbv8fIwmpJoqYOVjWnKGIQpil0YPhFjp77I14RioVfF3EyAFCgybZLQ/exec

---

General constraints for submissions: Please adhere to these rules to make our and your life easier! We will deduct points if you fail to do so.

- Your code should work with *Python 3.11*.
- You should only fill out the *TODO-gaps* and not change anything else in the code.
- Add comments to your code, to help us understand your solution.
- Your code should adhere to the PEP8 style guide. We allow line lengths of up to 120.
- While working on the exercise, push all commits to the `dev` branch (details in assignment 1). Only push your final results to the `main` branch, where they will be automatically tested in the cloud. If you push to `main` more than 3 times per exercise, we will deduct points.
- All provided unit tests have to pass: In your *GitHub* repository navigate to *Actions* → the *Autograding* run of your last commit to see which tests have passed. Autograding only shows Pass or Fail for each test; the tutors grade your submission and decide your points.
- `for` loops can be slow in Python, use vectorized `numpy` operations wherever possible (see assignment 1 for an example).
- Submit *a single PDF* named `submission.pdf`. At the top, write only your team name (no GitHub usernames, real names, email addresses or matriculation numbers). Add the answers and solution paths to all non-coding questions in the exercise. Do not leave answers to any questions as comments in the code. You can use LaTeX with the template `submission.tex` in your repository (see `setup.pdf`) or do it by hand.
- Please help us to improve the exercises by filling out and submitting the `feedback.md` file.
- We do not tolerate plagiarism. If you copy from other teams or the internet, you will get 0 points. Further action will be taken against repeat offenders!
- **Use of AI tools:** Using AI is not forbidden, but discouraged – solving the exercises by hand is the best way to learn. If you do use AI, you must adhere to the following rules, otherwise you will get **0 points** for the exercise:
  - **No auto-solving:** All code and answers must be reviewed by you and must not contain "AI artifacts".
  - **Declare usage:** If (and only if) your team used AI for an exercise, add a file named exactly `AI_USAGE.md` to the root of your repository stating which tools you used and for what. (Every grading run states whether this file was found.)
- Passing the exercises ($\geq 50\%$ in total) is a requirement for passing the course.

How to run the exercise and tests

- See the `setup.pdf` in exercise 1 / ILIAS for installation details.
- We always assume you run commands in the *root folder* of the exercise repository.
- If you use miniconda, do not forget to activate your environment with `conda activate mydlenv`
- Install the required packages with `pip install -r requirements.txt`
- Python files in the *root folder* of the repository contain the scripts to run the code.
- Python files in the `tests/` folder of the repository contain the tests that will be used to check your solution.
- Test everything at once with `python -m pytest`
- Run a single test with `python -m tests.test_something` (replace `something` with the test's name).
- To check your solution for the correct code style, run `pycodestyle --max-line-length 120 .`
- The scripts `runtests.sh` (Linux/Mac) or `runtests.bat` (Windows) can be used to run all the tests described above. If you are on Linux, you need execution rights to run `runtests.sh`.

This exercise focuses on optimization, we will:

- Calculate update steps of SGD with momentum by hand
- Implement the Adam optimizer
- Compare Adam, SGD, SGD with momentum
- Implement and analyze some learning rate schedules
- Analyze the effect of ill-conditioning and how it can be mitigated using momentum or preconditioning

1. Pen and Paper tasks

1) [2 points] Perform two update steps of SGD with momentum to the weights $\mathbf{w} = \begin{bmatrix} w_1 \\ w_2 \end{bmatrix}$ of the linear function $\hat{\mathbf{y}} = X\mathbf{w}$ optimizing the mean squared error (MSE). As in exercises 2 and 3, X contains one data point per row. The loss for one sample is given by $(\hat{y} - y)^2$, and, as for minibatch SGD in the lecture (Stochastic Gradient Descent (SGD) & Minibatch Sizes), the per-sample losses are summed and divided by the batch size. Use the momentum update from the lecture: $\mathbf{v} \leftarrow \beta\mathbf{v} - \alpha\hat{\mathbf{g}}$, $\mathbf{w} \leftarrow \mathbf{w} + \mathbf{v}$, where $\hat{\mathbf{g}}$ is the gradient of the loss with respect to $\mathbf{w}$.

   Given:

   the initial state $\mathbf{w} = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$, velocity $\mathbf{v} = \begin{bmatrix} 0 \\ 0 \end{bmatrix}$, data $X = \begin{bmatrix} 2 & -1 \\ -1 & 3 \end{bmatrix}$, target $\mathbf{y} = \begin{bmatrix} 3 \\ 1 \end{bmatrix}$, learning rate $\alpha = 0.2$, momentum strength $\beta = 0.8$ and batch size $B = 2$.

2) [2 points] Prove that in Adam the bias-corrected first moment estimate $\hat{s}$ (with s initialized to 0) is equal to the gradient in every step when that gradient remains constant.

   Hint: This can be done using mathematical induction.

2. Coding Tasks

In this coding exercise, you will implement an Adam optimizer and compare it with SGD and SGD with momentum. You will also implement two learning rate schedulers - one which uses piecewise constants and another which uses cosine annealing.

Please note that we extended the Module class in `lib/network_base.py` with a state (`self.training = True`) to determine whether we're training or evaluating the module. We also added two methods to toggle this state - `train()` and `eval()`. You will see these methods being used in the `train` function in `lib/training.py`.

1) [2 points] **Todo:** Implement the step function of the Adam optimizer (class `Adam` in file `lib/optimizers.py`) **Hint:** We provide an implementation of SGD in the same file.

   Run `python -m tests.test_adam_optimizer` to test your implementation.

2) [2 points] **Todo:** Train three models, one with Adam, one with SGD and one with SGD with momentum, for 10 epochs each. Each model should have one hidden layer with 30 hidden units followed by a ReLU activation. Start by implementing the `create_and_train_model` function and use it in `compare_optimizers` in file `lib/training.py` afterwards. Run `python -m tests.test_train_model` to test your implementation.

   Run `train_models.py` to run your code.

3) [2 points] **Todo:** Compare Adam vs SGD vs SGD with Momentum.

   Create two plots, one with the training loss curves and one with the training accuracy curves of the three models above (function `plot_learning_curves` in file `lib/plot.py`).

   Run `plot_trained_models.py` to run your code. Include the two plots in your `submission.pdf`.

4) [4 points] **Todo:** Implement the learning rate schedulers `PiecewiseConstantLR` and `CosineAnnealingLR` in file `lib/lr_schedulers.py`. You can use the provided class `LambdaLR` for this, which works similarly to the PyTorch LambdaLR, except that its function returns the learning rate itself, not a factor for the initial learning rate.

   PyTorch implements learning rate schedules as wrappers for the optimizer and requires the user to call `scheduler.step()` after each epoch. We've already done this for you in the training loop in function `train` in `lib/training.py`.

   Run `python -m tests.test_piecewise_scheduler` and `python -m tests.test_cosine_scheduler` to test your implementation.

5) [1 point] **Todo:** Plot learning rate schedules.

   To verify your implementation, plot the learning rate schedules with number of epochs on the x-axis and learning rate on the y-axis (function `plot_lr_schedules` in file `lib/plot.py`).

   Run `plot_lr_schedules.py` to run your code. Include the plot in your `submission.pdf`.

3. Experiments

We will now investigate how ill-conditioning affects gradient descent and two possible ways to mitigate its effects using momentum or preconditioning.

For these experiments, we aim to minimize a simple quadratic function, similar to the one presented in the lecture, of the form $f(\mathbf{x}) = \frac{1}{2}\mathbf{x}^\top Q\mathbf{x}$, where Q is a positive definite matrix.

1) [1 point] We will investigate the following cases:

   - Condition number = 1, momentum = 0.0, preconditioning = False
   - Condition number = 8, momentum = 0.0, preconditioning = False
   - Condition number = 8, momentum = 0.35, preconditioning = False
   - Condition number = 8, momentum = 0.0, preconditioning = True

   Note that for all cases, we use SGD as optimizer with the same learning rate and the same initial point. Finally, we stop after either 500 optimization steps or once we reach a value smaller than 1e-5 for the objective function we are aiming to minimize.

   Todo: Run `plot_ill_conditioned.py` for the cases mentioned above.

   Example for the last case:
   `python plot_ill_conditioned.py --condition_number 8 --momentum 0.0 --preconditioning`

   Some of the command line arguments that `python plot_ill_conditioned.py` accepts are:

   - `--condition_number`: Condition number of the quadratic function matrix. Default: 1.
   - `--momentum`: SGD momentum coefficient. Default: 0.0.
   - `--preconditioning`: Apply preconditioning matrix (inverse hessian of the quadratic function matrix) to the gradient. Default: False.

   Hint: Run `plot_ill_conditioned.py --help` for more options.

   Todo: Explain why the second experiment causes zigzag behaviour and why momentum and precondi-

tioning can mitigate this issue.

Include all the generated plots in your `submission.pdf`.

4. [1 bonus point] **Code Style**

On each exercise sheet, we will also be using `pycodestyle` to adhere to a common Python standard. `pycodestyle` checks your Python code against some common style conventions in PEP 8 and reports any deviations with specific error codes.

Your code will be automatically evaluated on submission (on push to `main`). You can run `pycodestyle --max-line-length=120 .` to manually evaluate your code before submission.

One bonus point will be awarded if there are no code style errors detected in your code by the `pycodestyle --max-line-length=120 .` command.

5. [1 bonus point] **Feedback**

Todo: Please give us feedback by filling out the `feedback.md` file.

- Problems you ran into?
- Helpful?
- Duration (hours)? For this, please follow the instructions in the `feedback.md` file.
- Other feedback?

This assignment is due on 25.11.2026 23:59 CET. Submit your solution for the tasks by uploading (`git push`) the PDF, txt file(s) and your code to your group's repository. The PDF has to include your team name only (no GitHub usernames or real names). Teams of at most 3 students are allowed.